VIT-AP UNIVERSITY

CSE2008 – **Operating Systems**

Dr. Venkata Rami Reddy Ch
Sr.Assistant Professor
School of Computer Science & Engineering
VIT-AP University

Module-3: Process Coordination and Deadlock

VIT-AP UNIVERSITY

- Process synchronization, critical-section problem, Peterson's solution, synchronization hardware, semaphores, classic problems of synchronization. System model, deadlock characterization, methods for handling deadlocks, deadlock prevention, deadlock avoidance, deadlock detection, recovery from deadlock.

Process synchronization

VIT-AP UNIVERSITY

- **Independent Process -** The process that does not share any shared variable, database, files, etc.
- **Cooperating Process -** The process that share file, variable, database, etc are the Cooperating Process.
- Concurrent access to shared data may result in **data inconsistency**,

Producer process

```
while (true) {
     /* produce an item in next_produced */

     while (counter == BUFFER_SIZE)
        ; /* do nothing */

     buffer[in] = next_produced;
     in = (in + 1) % BUFFER_SIZE;
     counter++;
}
```

Consumer process

```
while (true) {
     while (counter == 0)
        ; /* do nothing */

     next_consumed = buffer[out];
     out = (out + 1) % BUFFER_SIZE;
     counter--;

     /* consume the item in next_consumed */
}
```

- suppose that the value of the variable counter is currently 5 and that the producer and consumer processes concurrently execute the statements “counter++” and “counter--”.
- Following the execution of these two statements, the value of the variable counter may be 4, 5, or 6!
- The only correct result, though, is counter == 5, which is generated correctly if the producer and consumer executed in synchronized manner.

Process synchronization

VIT-AP UNIVERSITY

- We would arrive at this **incorrect state(data inconsistency)** because we allowed both processes to manipulate the variable count **concurrently**.
- A situation like this, leads to **race condition**
- Where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a **race condition**.
- To guard against the race condition above, we need to ensure that **only one process at a time** can be manipulating the variable count.
- To make such a guarantee, we require that the **processes be synchronized** in some way.

Process synchronization

VIT-AP UNIVERSITY

- Process synchronization is the task of synchronizing the execution of processes in such way that no two processes have to access the shared data at a time.
- In a multiprocessor system when multiple processes are running simultaneously, they attempt to access the shared data at a time.
- This can lead in inconsistency of shared data.
- That is the changes made by one process may not be reflected when other process accessed the shared data.
- In order to avoid data inconsistency ,the processes should be synchronized with each other.
- Process Synchronization, in which we allow **only one process** to **enter** and **manipulates the shared data** at a time.

Critical-Section Problem

VIT-AP UNIVERSITY

- Consider a system consisting of n processes {P0, P1,..., Pn−1}.
- A **Critical Section** is a **section of code common to cooperating processes**, where the process may be **accessing and updating shared data** (like variables, files, databases).
- The **Critical Section Problem** arises when **multiple processes access and update share resources** at the same time.
- The important feature of the system is that, when one process is executing in its critical section, no other process is allowed to execute in its critical section.
- That is, no two processes are executing in their critical sections at the same time.
- The critical-section problem is to **design a protocol** that the processes can be allowed to **share data without causing data inconsistencies**.

Critical-Section Problem

Critical Section

- The part of a process where shared variables or resources are accessed/modified.

Entry Section

- The entry section is the part of code where a process requests permission to enter its critical section

Exit Section

- The exit section is the part of the code where a process leaves its critical section and signals other processes that they can enter.

Remainder Section

- The rest of the process **outside the critical section**, which **does not use shared resources**..

General structure of process P_i

```
while (true) {
        entry section
            critical section
        exit section
            remainder section
}
```

Requirements of a CSP Solution

VIT-AP UNIVERSITY

- Any solution to the Critical-Section Problem must satisfy these **three conditions**:

Mutual Exclusion

- Only one process at a time can be executing in their critical section.

Progress

- If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely

Bounded waiting

- There exists a limit on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Peterson's solution

VIT-AP UNIVERSITY

- Peterson's Solution is a classic software-based solution to the critical section problem.
- software-based solutions are not guaranteed to work on modern computer architectures.
- Petersons solution is restricted to two processes that alternate execution between their critical sections and remainder sections.
- The processes are numbered P0 and P1.
- For convenience, when presenting Pi, we use Pj to denote the other process.
- Petersons solution requires the two processes to share two data items:
    - int turn;
    - boolean flag[2];
- The variable turn indicates whose turn it is to enter its critical section.
- That is, if turn == i, then process Pi is allowed to execute in its critical section.
- The flag array is used to indicate if a process is ready to enter its critical section.
- For example, if flag[i] is true, Pi is ready to enter its critical section.

Peterson's solution

VIT-AP UNIVERSITY

The structure of process Pi in Petersons solution:

```
do {
    flag[i] = TRUE;
    turn = j;
    while (flag[j] && turn == j);

        critical section

    flag[i] = FALSE;

        remainder section

} while (TRUE);
```

We now prove that this solution is correct.
We need to show that:

1. Mutual exclusion is preserved.
2. The progress requirement is satisfied.
3. The bounded-waiting requirement is met.

How mutual exclusion is preserved

VIT-AP UNIVERSITY

- Pi enters CS only if flag[j] == false OR turn == i
- Pj enters CS only if flag[i] == false OR turn == j
- Pi and Pj could not have successfully executed their while statements at the same time, since the value of turn can be either i or j but cannot be both.
- Because turn can only have one value at a time, and the while condition checks both flag[j] and turn, only one process can enter the critical section at a time, ensuring mutual exclusion.
- Let's assume **P0 and P1 try to enter simultaneously**:
  1. Both set flag[0] = true and flag[1] = true.
  2. Both set turn to the other process (turn = 1 by P0, turn = 0 by P1).
  3. **Only one value of turn "wins" last**, say turn = 1.
- Now check the while conditions:
  - **P0**: while(flag[1] == true && turn == 1) → true → **P0 waits**
  - **P1**: while(flag[0] == true && turn == 0) → false → **P1 enters CS**
- So, **only one process enters CS**, the other waits. ☑

How progress is preserved

VIT-AP UNIVERSITY

How Progress is Ensured

Case-1: If Pj is not ready to enter critical section then

- flag[j] == false
- ☑ Pi can enter its critical section.

Case-2: If Pj set flag[j] to true **to enter CS and is in its while loop**

- Two possibilities for turn:
  - turn == i → Pi enters CS
  - turn == j → Pj enters CS

How Bounded Waiting is preserved

VIT-AP UNIVERSITY

- Each process waits at most **one entry of the other process** before entering CS.

- Suppose turn == j and Pi is waiting and Pj in **CS**.
- Once Pj **exits CS**, it sets flag[j] = false then Pi can now enter CS.
- Even if Pj wants to re-enter CS, it must set turn = i to let Pi go first.

☑ **Result:** Pi cannot be delayed indefinitely — it waits at most **one turn of Pj**.

Hardware Support for Synchronization

VIT-AP UNIVERSITY

- Software-based solutions are not guaranteed to work on modern computer architectures.
- Many modern computer systems provide special hardware instructions such as test_and_set() , compare_and_swap() hardware instructions to solve the critical-section problem in a relatively simple manner.

test_and_set()

VIT-AP UNIVERSITY

How it Works

The instruction does two things **atomically**:

- **Reads** the current value of a memory location (say a lock variable).
- **Sets** that memory location to 1 (locked).

Then it **returns the old value**.

```
boolean test_and_set(boolean *target) {
   boolean rv = *target;
   *target = true;

   return rv;
}
```

Mutual-exclusion implementation with test_and_set()

VIT-AP UNIVERSITY

```
boolean test_and_set(boolean *target) {
   boolean rv = *target;
   *target = true;

   return rv;
}
```

The structure of process Pi:

- variable **lock**, initialized to **false**

  for first process

  lock=false

```
do {
   while (test_and_set(&lock))
      ; /* do nothing */

      /* critical section */

   lock = false;

      /* remainder section */
} while (true);
```

Mutual-exclusion implementation with test_and_set()

VIT-AP UNIVERSITY

Why It Ensures Mutual Exclusion

If two processes call test_and_set(&lock) at the same time:

- Only **one** will see the old value false (0) and set it to true (1).
- The other(s) will see true and will spin in the loop.

So **only one process enters the critical section at a time**. ☑

compare_and_swap()

VIT-AP UNIVERSITY

How it Works

The instruction does three things **atomically**:

1. **Reads** the current value of a memory location (*addr).
2. **Compares** it with an **expected value**.
    1. If equal → **swaps** it with a **new value**.
    2. If not equal → leaves it unchanged.

Finally, it **returns the old value** (the value that was in memory before the operation).

- compare_and_swap() instruction:

```
int compare_and_swap(int *value, int expected, int new_value) {
   int temp = *value;

   if (*value == expected)
      *value = new_value;

   return temp;
}
```

Mutual exclusion with the compare_and_swap()


```
int compare_and_swap(int *value, int expected, int new_value) {
   int temp = *value;

   if (*value == expected)
      *value = new_value;

   return temp;
}
```

The structure of process Pi:

- The first process that invokes compare and swap() will set lock to **0**.
- lock=0

```
while (true) {
   while (compare_and_swap(&lock, 0, 1) != 0)
      ; /* do nothing */

      /* critical section */

   lock = 0;

      /* remainder section */
}
```

Mutual exclusion with the compare_and_swap()

Why It Ensures Mutual Exclusion

- Only **one process** will successfully change lock from 0 → 1.
- Others will see lock != 0 and spin until it’s released.

Semaphores

- In 1965, Dijkstra proposed a new and very significant technique for managing concurrent processes by using the value of a simple integer variable to synchronize the progress of interacting processes.
- A semaphore is a simple integer variable used to control the access of a shared resource by multiple processes.
- When one process modifying the semaphore value, no other process can simultaneously modify that same semaphore value.
- A **semaphore** S is accessed only through two standard atomic operations: wait() and signal().
- The wait() operation was originally termed P (Prolaag) and signal() was originally called V (Verhoog).

Semaphores

wait():

- It decrements the semaphore value by 1.

Definition:

```
wait(S) {
while (S <= 0)
; // busy wait
S--;
}
```

signal():

it increments the semaphore value by 1.

Definition :

```
signal(S) {
S++;
}
```

Types of Semaphores

Semaphores are mainly of two types:

1. Binary semaphore
2. Counting semaphore

Binary semaphore:

- It is a semaphore whose value is either 0 or 1.
- Binary semaphore can be used to control access to a single resource.
- It is also called mutex locks to provide mutual exclusion.

Counting semaphore:

- It is a semaphore whose value is between 0 or positive number(no of resources available)
- Counting semaphores can be used to control access to a given resource consisting of a finite number of instances.

Semaphore Usage

- The semaphore is initialized to the number of resources available.
- Each process that wishes to use a resource performs a wait() operation on the semaphore.
- When a process releases a resource, it performs a signal() operation.
- When the count for the semaphore goes to 0, all resources are being used.
- After that, processes that wish to use a resource will block until the count becomes greater than zero.

Semaphore Implementation

Mutual-exclusion implementation with semaphore:

```
mutex=1
do {
wait (mutex) ;
// critical section
signal(mutex);
// remainder section
} while (TRUE);
```

```
wait(S) {
while (S <= 0)
; // busy wait
S--;
}
```

- The main disadvantage of the semaphore definition given here is that it requires **busy waiting**.
- While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the wait operation.
- Busy waiting wastes CPU cycles.

Semaphore Implementation without busy waiting

- To overcome the need for busy waiting, we can modify the definition of the wait() and signal() semaphore operations.
- When a process executes the wait () operation and finds that the semaphore value is not positive, it must wait.
- However, rather than engaging in busy waiting, the process can *block* itself.
- The block operation places a process into a waiting queue, and the state of the process is switched to the waiting state.
- A process that is blocked, should be restarted when some other process executes a signal() operation.
- The process is restarted by a wakeup () operation, which changes the process from the waiting state to the ready state.

Semaphore Implementation without busy waiting

- To implement semaphores under this definition, we define a semaphore as a "C' struct:

```
typedef struct {
int value;
struct process *list;
} semaphore;
```

- Each semaphore has an integer value and a list.
- When a process must wait on a semaphore, it is added to the list.

Semaphore Implementation without busy waiting

- In the modified wait(), we check **S->value < 0**.

•If it's **negative**, it means resources are unavailable and the current process must wait.

•S->value = -2 means 2 processes are in the **waiting queue**.

if (S->value <= 0)

- This condition means: before increment, the semaphore's value was **negative** (so processes were waiting).

remove a process P from S->list;

- From the queue select one process.

wakeup(P);

•Allow that process to resume execution

modified wait():

```
wait(semaphore *S) {
S->value--;
if (S->value < 0) {
add this process to S->list;
block();
}  }
```

modified signal():

```
signal(semaphore *S) {
S->value++;
if (S->value <= 0) {
remove a process P from  S->list;
wakeup(P);
} }
```

Deadlocks and Starvation in semaphores

- The implementation of a semaphore with a waiting queue may result in a situation where two or more processes are waiting indefinitely called deadlock.

- Suppose that P0 executes wait(S)and then P1 executes wait(Q).
- When P0 executes wait(Q), it must wait until P1 executes signal(Q).
- Similarly, when P1 executes wait(S), it must wait until P0 executes signal(S).
- Since these signal() operations cannot be executed, P0 and P1 are deadlocked.
- Another problem related to dead locks is starvation, a situation in which processes wait indefinitely with in the semaphore.

P_0	P_1
wait(S);	wait(Q);
wait(Q);	wait(S);
.	.
.	.
.	.
signal(S);	signal(Q);
signal(Q);	signal(S);

Classical Problems of Synchronization

1. Bounded-Buffer Problem
2. Readers and Writers Problem
3. Dining-Philosophers Problem

The Bounded-Buffer Problem(Producer–Consumer)

Producer generates data items and puts them into a buffer.
Consumer takes items from the buffer.
The buffer has **limited size (N slots)**.

Constraints:

- Producer must wait if buffer is **full**.
- Consumer must wait if buffer is **empty**.
- Mutual exclusion is required so two processes don’t update the buffer simultaneously.

The Bounded-Buffer Problem using Semaphores

Semaphores Used:

We use **three semaphores**:

semaphore mutex = 1;
semaphore empty = n;
semaphore full = 0

mutex (binary semaphore)

- Ensures **mutual exclusion** when accessing buffer.

empty (counting semaphore)

- Counts how many **empty slots** are available.

full (counting semaphore)

- Counts how many **slots** are filled.

The Bounded-Buffer Problem

The structure of the producer process:

```
while (true) {
   wait(empty);
    wait(mutex);
        ...
/* add item to the buffer */
        ...
    signal(mutex);
    signal(full);
}
```

- Before inserting → checks empty.
- If no space (empty == 0), producer blocks.
- Uses mutex to ensure only one producer/consumer modifies buffer at a time.
- After inserting → signals mutex and full.

The Bounded-Buffer Problem

The structure of the Consumer process

```
while (true) {
    wait(full);
    wait(mutex);
        ...
    /* take an item from buffer */
        ...
    signal(mutex);
    signal(empty);
}
```

- Before removing → checks full.
- If no item (full == 0), consumer blocks.
- Uses mutex for exclusive access.
- After removing → signals mutex & empty.

The Readers-Writers Problem

- A database is to be shared among several concurrent processes.
  - **Readers** – only read the data from data base; they do ***not*** perform any updates
  - **Writers** – Update the data in database, requiring exclusive access.

Constraints:

- **Writers require exclusive access** → no other readers or writers can access while writing.
- Any number of readers may read the data simultaneously.
- If a write is updating the data no reader may read it.

The Readers-Writers Problem

- In the solution to the readers–writers problem, the reader processes share the following data structures:
    - semaphore rw_mutex = 1;
    - semaphore mutex = 1;
    - int read_count = 0;
- The mutex semaphore is used to ensure mutual exclusion when the variable read_count is updated.
- The read_count variable keeps track of how many processes are currently reading the object.
- The semaphore rw_mutex functions as a mutual exclusion semaphore for the writers.
- It is also used by the first or last reader that enters or exits the critical section.
- It is not used by readers who enter or exit while other readers are in their critical sections.

The Readers-Writers Problem

The structure of a writer process

```
while (true) {
     wait(rw_mutex);
          ...
     /* writing is performed */
          ...
    signal(rw_mutex);
}
```

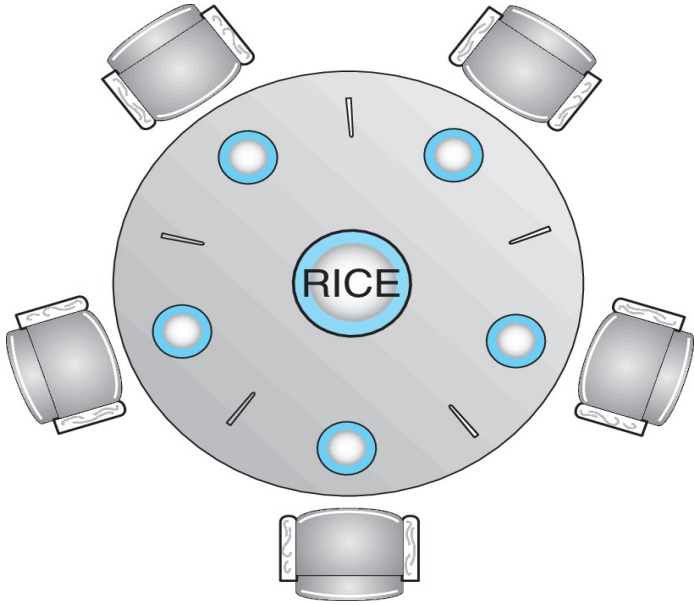
The Readers-Writers Problem

The structure of a reader process

```
while (true){
     wait(mutex);
     read_count++;
     if (read_count == 1)     /* first reader */
          wait(rw_mutex);
          signal(mutex);
         ...
           /* reading is performed */
         ...
     wait(mutex);
          read_count--;
          if (read_count == 0)    /* last reader */
          signal(rw_mutex);
     signal(mutex);
}
```

The Dining-Philosophers Problem

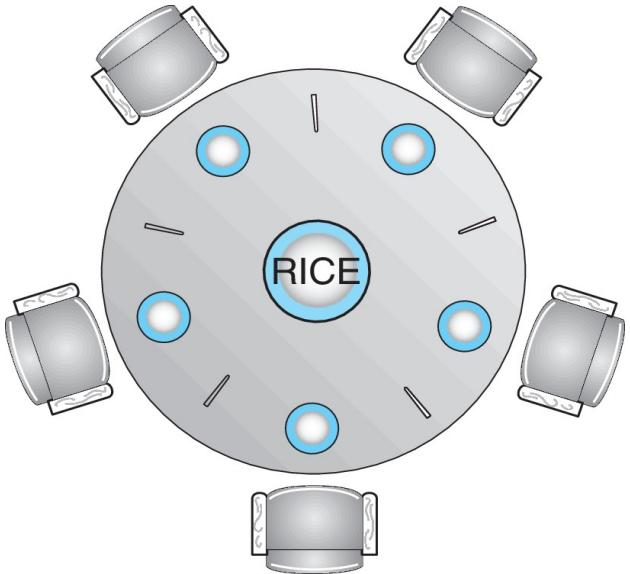
- Five philosophers sit around a **circular table**.
- Each philosopher has a **chair** and alternates between two activities: **thinking** and **eating**.
- A bowl of rice is placed at the **center** of the table.
- Between each pair of philosophers is a **single chopstick**, so there are **five chopsticks** in total.
- To eat, a philosopher must acquire **both the left and right chopsticks**.
- After eating, they **put down both chopsticks** and return to **thinking**.
- A philosopher may **pick up** only **one chopstick** at **a time**.
- Obviously, she **cannot pickup** a chopstick that is **already in the hand of a neighbor**.



The Dining-Philosophers Problem

- One simple solution is to represent each chopstick with a semaphore.
- A philosopher tries to grab a chopstick by executing a wait() operation on that semaphore.
- She releases her chopsticks by executing the signal() operation on the appropriate semaphores

- Thus, the shared data are
  - semaphore chopstick[5];
- where all the elements of chopstick are initialized to 1.



The Dining-Philosophers Problem

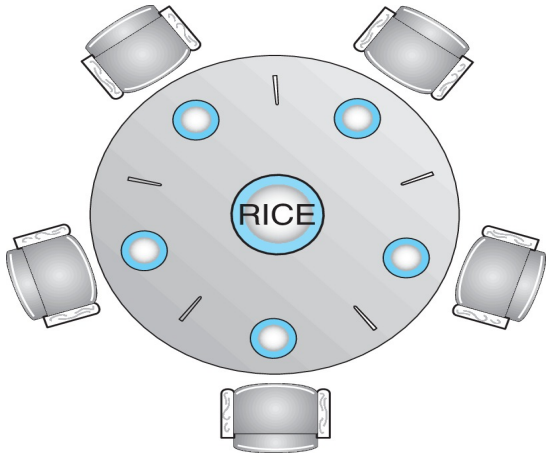
The structure of Philosopher i :

```
semaphore chopstick[5] = {1, 1, 1, 1, 1};
      while (true){
          wait(chopstick[i] );                    // pick left
         wait(chopStick[ (i + 1) % 5] );          // pick right

          /* eat for awhile */

         signal (chopstick[i] );                  // put left
         signal (chopstick[ (i + 1) % 5] );       // put right

          /* think for awhile */
      }
```



The Dining-Philosophers Problem

What is the problem with this algorithm?

- Although this solution guarantees that no two neighbors are eating simultaneously, it could create a deadlock.
- Suppose that all five philosophers become hungry at the same time and each grabs her left chopstick.
- All the elements of chopstick will now be equal to 0.
- When each philosopher tries to grab her right chopstick, she will be delayed forever.

Deadlock in Operating System

- A deadlock is a situation where a set of processes gets permanently blocked because each process is waiting for a resource held by another process, and none of them can proceed.

How Does Deadlock Occur in OS?

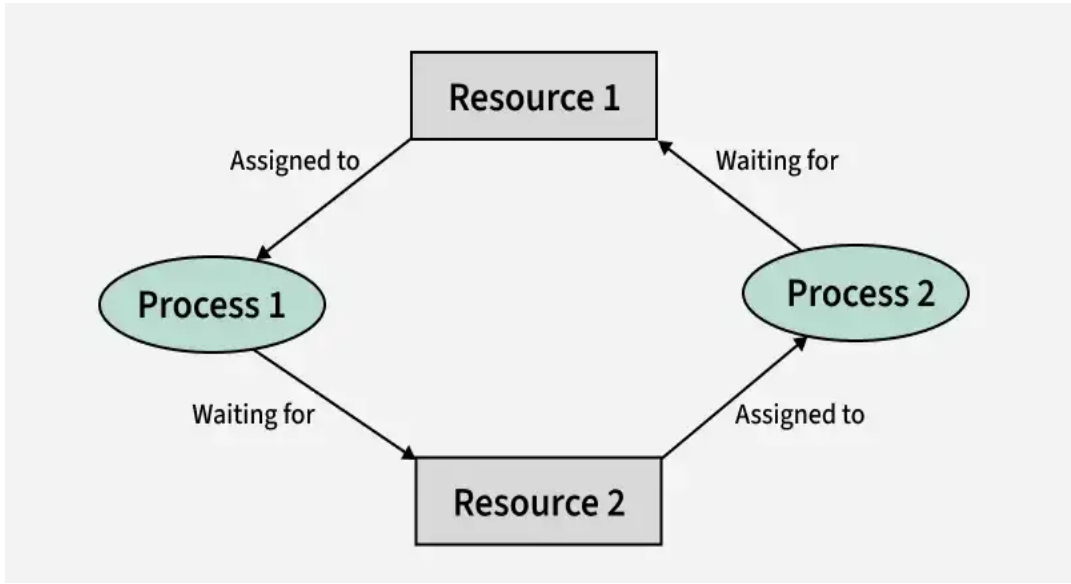
- Deadlock arises when processes hold some resources while waiting for others.

Example:

•Process P1 holds Resource R1 and requests R2.

•Process P2 holds Resource R2 and requests R1.

Neither process can proceed causing a deadlock



System Model

- System consists of resources
- Resource types $R_1, R_2, \ldots, R_m$
  - *CPU cycles, memory space, I/O devices*
- Each resource type R_i has W_i instances.
- If a system has two CPUs, then the resource type CPU has two instances.
- The number of resources requested may not exceed the total number of resources available in the system.
- A process may utilize a resource in only the following sequence:

Request: The process requests the resource. If the request cannot be granted immediately (for example, if the resource is being used by another process), then the requesting process must wait until it can acquire the resource.

Use: The process can operate on the resource.

Release: The process releases the resource.

Resource-Allocation Graph

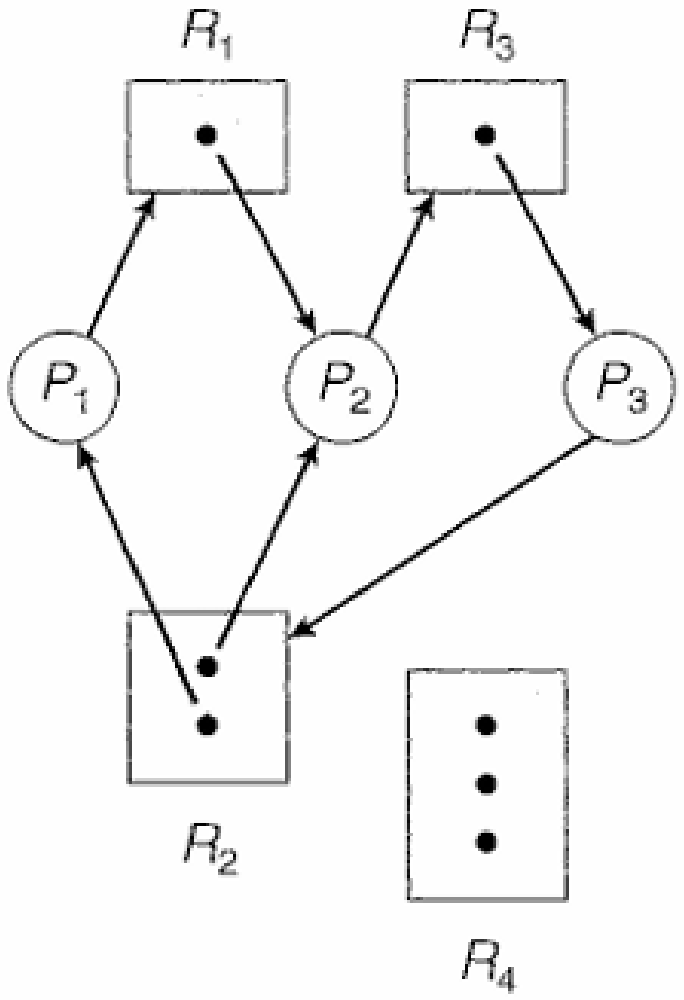
- Deadlocks can be described more precisely in terms of a directed graph called Resource-Allocation Graph
- This graph consists of a set of vertices V and a set of edges E.
- V is partitioned into two types:
  - P == { P1, P2, ... , Pn}, the set consisting of all the processes in the system. $R = \{R_1, R_2, \ldots, R_m\}$, the set consisting of all resource types in the system

- **request edge** – directed edge $P_i \rightarrow R_j$

- **assignment edge** – directed edge $R_j \rightarrow P_i$

Resource Allocation Graph with a Deadlock

- One instance of R_1
- Two instances of R_2
- One instance of R_3
- Three instance of R_4
- P_1 holds one instance of R_2 and is waiting for an instance of R_1
- P_2 holds one instance of R_1, one instance of R_2, and is waiting for an instance of R_3
- P_3 holds one instance of R_3 and waiting for one instance of R_2



- Dead lock Occurred.

Deadlock Avoidance

Safe State

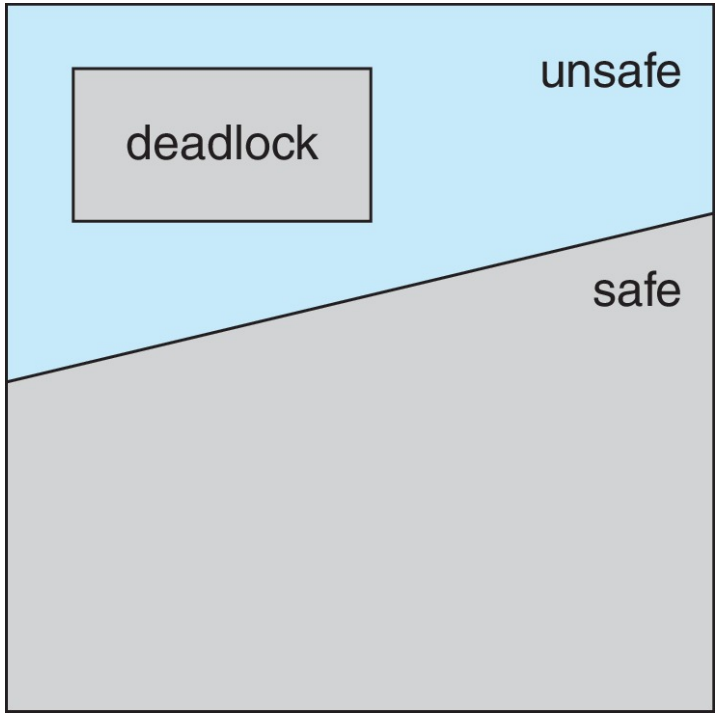
- A state is safe if the system can allocate resources to each process up to its maximum in some order and still avoid a deadlock.
- More formally, a system is in a safe state only if there exists a safe sequence.
- To illustrate, we consider a system with twelve magnetic tape drives and three processes:
- there are three free tape drives
- At time t0, the system is in a safe state. The sequence <P1,PO,P2>satisfies the safety condition.

- Process P1 can immediately be allocated all its tape drives and then return them (5 available tape drives);
- then process Po can get all its tape drives and return them (ten available tape drives);
- and finally process P2 can get all its tape drives and return them (all twelve tape drives available).

	Maximum Need	Holding
P0	10	5
P1	4	2
P2	9	2

Deadlock Avoidance

- If a system is in safe state $\Rightarrow$ no deadlocks
- If a system is in unsafe state $\Rightarrow$ possibility of deadlock
- Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state



Banker’s Algorithm for Deadlock Avoidance

- Banker's Algorithm is a resource allocation and deadlock avoidance algorithm used in operating systems.
- It ensures that a system remains in a safe state by carefully allocating resources to processes while avoiding unsafe states that could lead to deadlocks.

Data Structures for the Banker's Algorithm

Let *n* = number of processes, and *m* = number of resources types.

Available – It is a vector of length "m" that indicates the number of available resources of each type in the system. If Available [j] = k, then "k" be the instances of resource type R_j available in the system.

Max – It is a [n x m] matrix that defines the maximum resource demand of each process. When Max [I, j] = k, then a process P_i may request maximum k instances of resource type, R_j.

Allocation – – It is also a [n x m] matrix that defines the number of resources of each type which are currently allocated to each process. Therefore, if Allocation [i, j] = k, then a process P_i is currently allocated "k" instances of resource type, R_j.

Data Structures for the Banker's Algorithm

Need – It is an [n × m] matrix that represents the remaining resource need of each process. When Need [i, j] = k, then a process P_i may need "k" more instances of resource type R_j to accomplish the assigned task.

Finish – It is a matrix of length "m". It includes a Boolean value, i.e. TRUE or FALSE to indicate whether a process has been allocated to the needed resources, or all resources have been released after finishing the assigned work.

Also, it is to be noted that,

Need [i, j] = Max [i, j] – Allocation [i, j]

The baker's algorithm is a combination of two algorithms namely, **Safety Algorithm** and **Resource Request Algorithm.** These two algorithms together control the processes and avoid dead lock in a system.

Safety Algorithm

- A safety algorithm is an algorithm used to find whether or not a system is in its safe state.

1. Let ***Work*** and ***Finish*** be vectors of length *m* and *n*, respectively. Initialize:

   Work* = *Available

   ***Finish* [*i*] = *false* for *i* = 0, 1, …, *n*- 1**

2. Find an ***i*** such that both:

   (a) ***Finish* [*i*] = *false***

   (b) $Need_i \leq Work$

   If no such ***i*** exists, go to step 4

3. $Work = Work + Allocation_i$

   Finish*[*i*] = *true

   go to step 2

4. If ***Finish* [*i*] == *true*** for all ***i***, then the system is in a safe state

Resource-Request Algorithm for Process P_i

This algorithm determines how a system behaves when a process request for each type of resource in the system.

Let us consider a request vector Requesti for a process P_i. When Requesti [j] = k, then the process P_i requires k instances of resource type R_j.

When a process Pi requests for resources, the following sequence is followed −

1. If $\boldsymbol{Request_i \leq Need_i}$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If $\boldsymbol{Request_i \leq Available}$, go to step 3. Otherwise, $\boldsymbol{P_i}$ must wait, since resources are not available
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$$\boldsymbol{Available = Available - Request_i;}$$

$$\boldsymbol{Allocation_i = Allocation_i + Request_i;}$$

$$\boldsymbol{Need_i = Need_i - Request_i;}$$

- If safe $\Rightarrow$ the resources are allocated to $\boldsymbol{T_i}$
- If unsafe $\Rightarrow$ $\boldsymbol{T_i}$ must wait, and the old resource-allocation state is restored

Example:

Considering a system with five processes P_0 through P_4 and three resources of type A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t_0 following snapshot of the system has been taken:

Process	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Q.1: What will be the content of the Need matrix?

Need [i, j] = Max [i, j] – Allocation [i, j]
So, the content of Need Matrix is:

Process	Need		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

Q.2: Is the system in a safe state? If Yes, then what is the safe sequence?

Applying the Safety algorithm on the given system,

m=3, n=5 Step 1 of Safety Algo

Work = Available

Work = | 3 | 3 | 2 |

	0	1	2	3	4
Finish =	false	false	false	false	false

For i = 0 Step 2

$Need_0$ = 7, 4, 3 7,4,3 3,3,2

Finish [0] is false and $Need_0$ > Work

So P_0 must wait But Need ≤ Work

For i = 1 Step 2

$Need_1$ = 1, 2, 2 1,2,2 3,3,2

Finish [1] is false and $Need_1$ < Work

So P_1 must be kept in safe sequence

Step 3

3, 3, 2 2, 0, 0

Work = Work + $Allocation_1$

	A	B	C
Work =	5	3	2

	0	1	2	3	4
Finish =	false	true	false	false	false

For i = 2 Step 2

$Need_2$ = 6 , 0, 0 6, 0, 0 5,3, 2

Finish [2] is false and $Need_2$ > Work

So P_2 must wait

For i=3 Step 2

$Need_3$ = 0, 1, 1

Finish [3] = false and $Need_3$ < Work (0, 1, 1 < 5, 3, 2)

So P_3 must be kept in safe sequence

Step 3

Work = Work + $Allocation_3$ (5, 3, 2 + 2, 1, 1)

	A	B	C
Work =	7	4	3

	0	1	2	3	4
Finish =	false	true	false	true	false

For i = 4 Step 2

$Need_4$ = 4, 3, 1

Finish [4] = false and $Need_4$ < Work (4, 3, 1 < 7, 4, 3)

So P_4 must be kept in safe sequence

Step 3

Work = Work + $Allocation_4$ (7, 4, 3 + 0, 0, 2)

	A	B	C
Work =	7	4	5

	0	1	2	3	4
Finish =	false	true	false	true	true

For i = 0 Step 2

$Need_0$ = 7, 4, 3

Finish [0] is false and Need < Work (7, 4, 3 < 7, 4, 5)

So P_0 must be kept in safe sequence

Step 3

Work = Work + Allocation$_0$ (7, 4, 5 + 0, 1, 0)

	A	B	C
Work =	7	5	5

	0	1	2	3	4
Finish =	true	true	false	true	true

Step 2

For i = 2

Need$_2$ = 6 , 0, 0

Finish [2] is false and Need$_2$ < Work (6, 0, 0 < 7, 5, 5)

So P_2 must be kept in safe sequence

Step 3

Work = Work + Allocation$_2$ (7, 5, 5 + 3, 0, 2)

	A	B	C
Work =	10	5	7

	0	1	2	3	4
Finish =	true	true	true	true	true

Step 4

Finish [i] = true for $0 \leq i \leq n$

Hence the system is in Safe state

The safe sequence is P_1, P_3, P_4, P_0, P_2

Q.3: What will happen if process P_1 requests one additional instance of resource type A and two instances of resource type C?

A B C

$Request_1$ = 1, 0, 2

To decide whether the request is granted we use Resource Request algorithm

Step 1

1, 0, 2 1, 2, 2

$Request_1 < Need_1$

Step 2

1, 0, 2 3, 3, 2

$Request_1 < Available$

Step 3

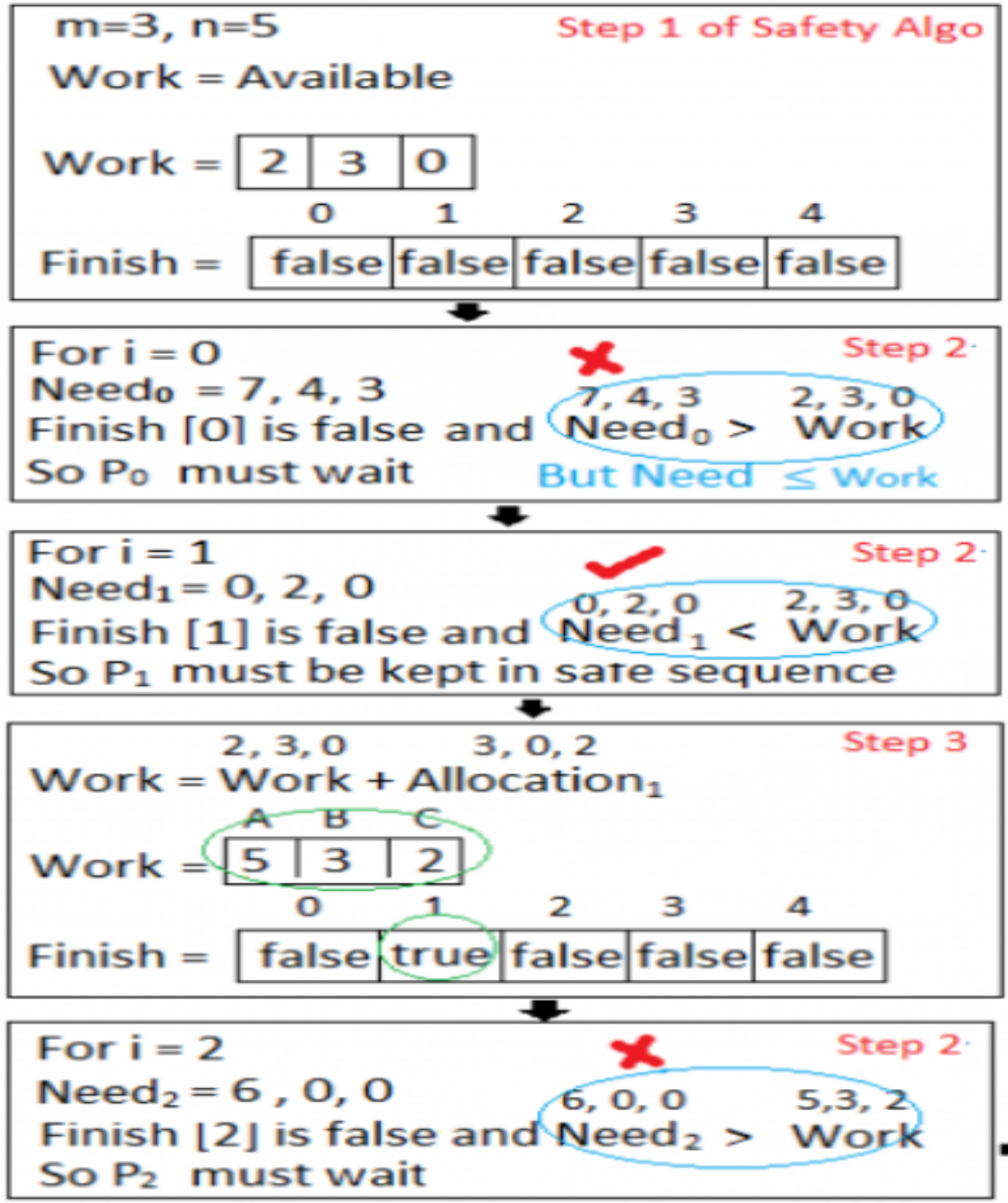
$Available = Available - Request_1$

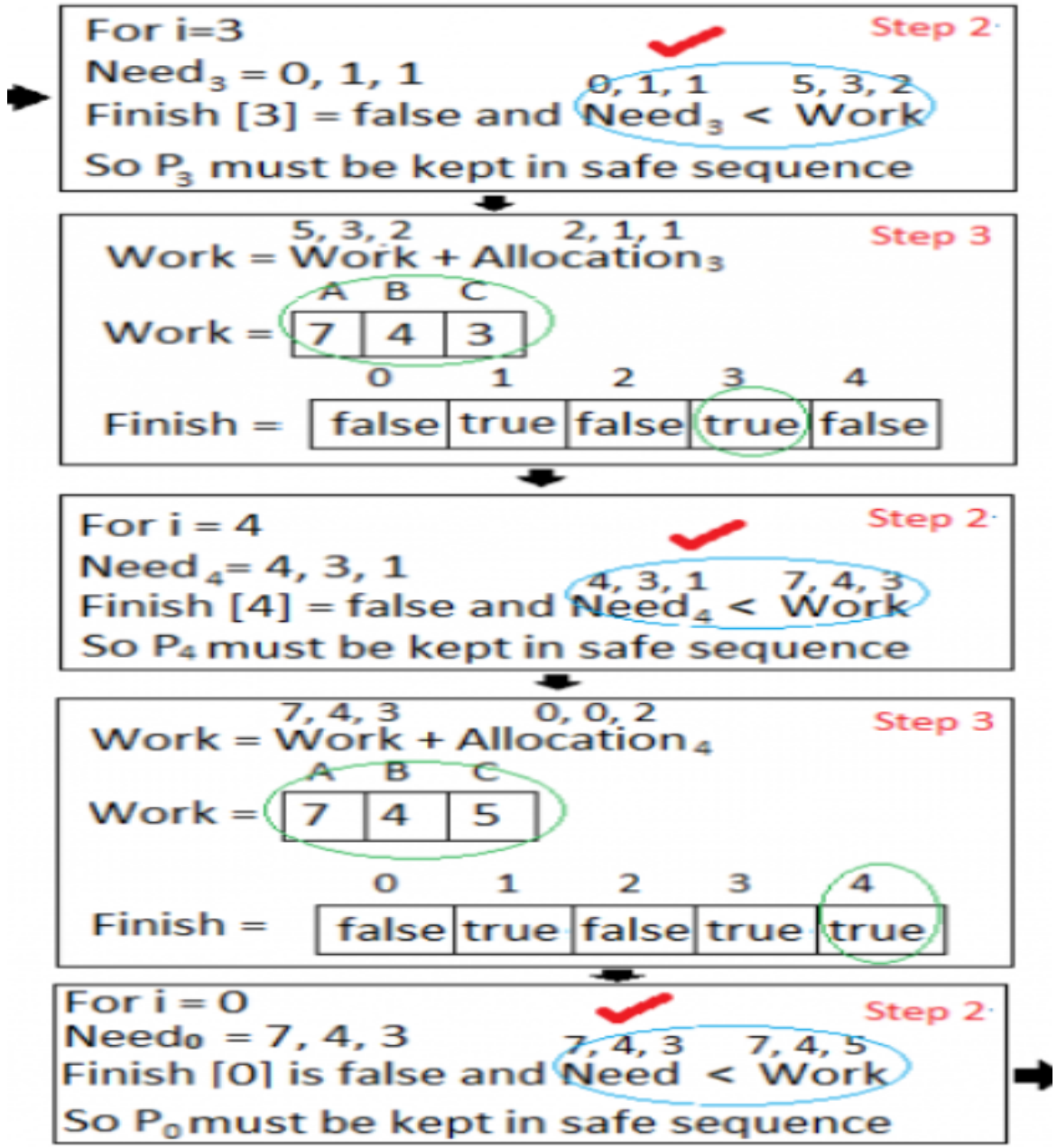
$Allocation_1 = Allocation_1 + Request_1$

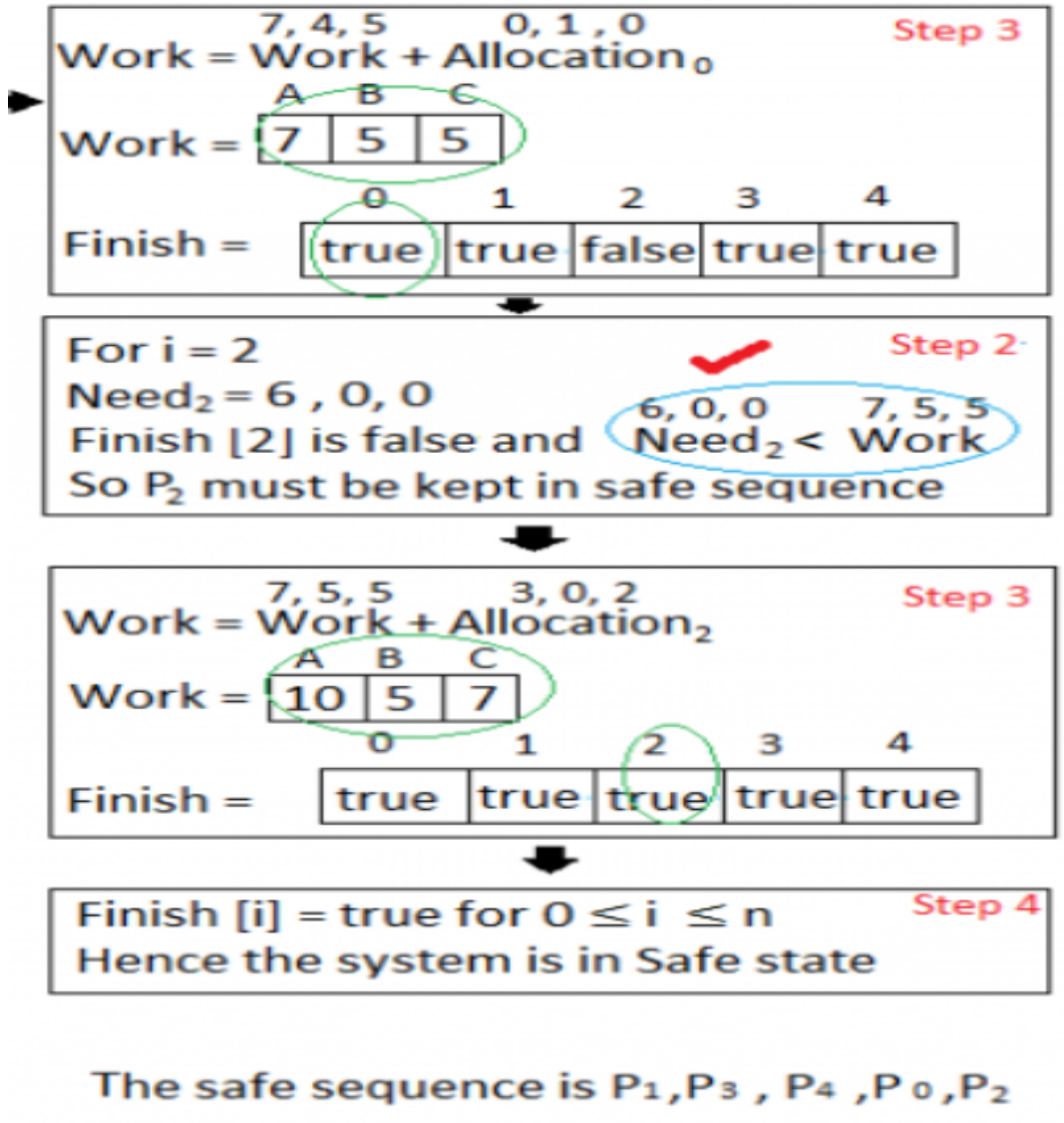
$Need_1 = Need_1 - Request_1$

Process	Allocation	Need	Available
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

We must determine whether this new system state is safe. To do so, we again execute Safety algorithm on the above data structures.


m=3, n=5
Step 1 of Safety Algo
Work = Available
Work = 2 3 0
0 1 2 3 4
Finish = false false false false false
For i = 0
Need0 = 7, 4, 3
Finish [0] is false and
So P0 must wait
7, 4, 3 Need0 > 2, 3, 0 Work
But Need ≤ Work
Step 2
For i = 1
Need1 = 0, 2, 0
Finish [1] is false and
So P1 must be kept in safe sequence
0, 2, 0 Need1 < 2, 3, 0 Work
Step 2
2, 3, 0
3, 0, 2
Work = Work + Allocation1
A B C
Work = 5 3 2
0 1 2 3 4
Finish = false true false false false
Step 3
For i = 2
Need2 = 6 , 0, 0
Finish [2] is false and
So P2 must wait
6, 0, 0 Need2 > 5,3, 2 Work
Step 2


For i=3
Step 2
Need3 = 0, 1, 1
0, 1, 1 5, 3, 2
Finish [3] = false and Need3 < Work
So P3 must be kept in safe sequence
5, 3, 2 2, 1, 1
Step 3
Work = Work + Allocation3
A B C
Work = 7 4 3
0 1 2 3 4
Finish = false true false true false
For i = 4
Step 2
Need4= 4, 3, 1
4, 3, 1 7, 4, 3
Finish [4] = false and Need4 < Work
So P4 must be kept in safe sequence
7, 4, 3 0, 0, 2
Step 3
Work = Work + Allocation4
A B C
Work = 7 4 5
0 1 2 3 4
Finish = false true false true true
For i = 0
Step 2
Need0 = 7, 4, 3
7, 4, 3 7, 4, 5
Finish [0] is false and Need < Work
So P0 must be kept in safe sequence



Hence the new system state is safe, so we can immediately grant the request for process P_1.

Consider the following snapshot of a system:

	Allocation	*Max*	*Available*
	A B C D	*A B C D*	*A B C D*
T_0	0 0 1 2	0 0 1 2	1 5 2 0
T_1	1 0 0 0	1 7 5 0	
T_2	1 3 5 4	2 3 5 6	
T_3	0 6 3 2	0 6 5 2	
T_4	0 0 1 4	0 6 5 6	

Answer the following questions using the banker's algorithm:

a. What is the content of the matrix ***Need***?

b. Is the system in a safe state?

c. If a request from thread T_1 arrives for (0,4,2,0), can the request be granted immediately?

A)

Max	Allocation
T0: (0,0,1,2)	T0: (0,0,1,2)
T1: (1,7,5,0)	T1: (1,0,0,0)
T2: (2,3,5,6)	T2: (1,3,5,4)
T3: (0,6,5,2)	T3: (0,6,3,2)
T4: (0,6,5,6)	T4: (0,0,1,4)

Need = Max − Allocation

T0: (0,0,0,0)

T1: (0,7,5,0)

T2: (1,0,0,2)

T3: (0,0,2,0)

T4: (0,6,4,2)

Initial Available = Work = (1,5,2,0)

Finish flags: all false

B— Safety check step-by-step (find a safe sequence)

Start: `Work = (1,5,2,0), Finish =[F,F,F,F,F].`

1.Check T0: Need = (0,0,0,0) ≤ Work (1,5,2,0) → **yes**.

1. `Work ← Work + Alloc(T0) = (1,5,2,0) + (0,0,1,2) = (1,5,3,2)`
2. Sequence so far: **T0**

2.Check T1: Need (0,7,5,0) ≤ Work (1,5,3,2)? → **No** (7 > 5). Skip.

3.Check T2: Need (1,0,0,2) ≤ Work (1,5,3,2) → **Yes**.

1. `Work ← (1,5,3,2) + (1,3,5,4) = (2,8,8,6)`
2. Sequence: **T0 ,T2**

4.Check T3: Need (0,0,2,0) ≤ Work (2,8,8,6) → **Yes**.

1. `Work ← (2,8,8,6) + (0,6,3,2) = (2,14,11,8)`
2. Sequence: **T0 , T2 , T3**

5.Check T4: Need (0,6,4,2) ≤ Work (2,14,11,8) → **Yes**.

1. `Work ← (2,14,11,8) + (0,0,1,4) = (2,14,12,12)`
2. Sequence: **T0 , T2 , T3,T4**

6.Check T1 (last): Need (0,7,5,0) ≤ Work (2,14,12,12) → **Yes**.

1. `Work ← (2,14,12,12) + (1,0,0,0) = (3,14,12,12)`
2. `Finish = [T,T,T,T,T]`
3. Final sequence: **T0 , T2 , T3,T4,T1**

All processes can finish → **system is in a safe state**. Safe sequence shown above.

C— Now: T1 requests (0,4,2,0). Check step-by-step if it can be granted immediately

1. Check Request ≤ Need(T1):

•Request = (0,4,2,0)

•Need(T1) = (0,7,5,0) → (0≤0, 4≤7, 2≤5, 0≤0) → **OK**

2. Check Request ≤ Available (current):

•Available = (1,5,2,0) → (0≤1, 4≤5, 2≤2, 0≤0) → **OK**

Since both checks pass, tentatively grant and perform safety test with the updated state.

Tentative grant (temporary):

•`Available' = Available − Request = (1,5,2,0) − (0,4,2,0) = (1,1,0,0)`

•`Alloc'(T1) = Alloc(T1) + Request = (1,0,0,0) + (0,4,2,0) = (1,4,2,0)`

•`Need'(T1) = Need(T1) − Request = (0,7,5,0) − (0,4,2,0) = (0,3,3,0)`

Other Alloc/Need unchanged.

Now run safety check with `Work = Available' = (1,1,0,0)` and Finish = all false.

Safety check after tentative grant:

Start: `Work = (1,1,0,0),Finish = [F,F,F,F,F].`

1.T0: Need (0,0,0,0) ≤ Work (1,1,0,0) → **Yes**.

`1.Work ← (1,1,0,0) + Alloc(T0) (0,0,1,2) = (1,1,1,2)`

2. Seq: **T0**

2.T2: Need (1,0,0,2) ≤ Work (1,1,1,2) → **Yes**.

`1.Work ← (1,1,1,2) + (1,3,5,4) = (2,4,6,6)`

2. Seq: **T0 ,T2**

3.T3: Need (0,0,2,0) ≤ Work (2,4,6,6) → **Yes**.

`1.Work ← (2,4,6,6) + (0,6,3,2) = (2,10,9,8)`

2. Seq: **T0 , T2 , T3**

4.T4: Need (0,6,4,2) ≤ Work (2,10,9,8) → **Yes**.

`1.Work ← (2,10,9,8) + (0,0,1,4) = (2,10,10,12)`

1. Seq: **T0 , T2 , T3 , T4**

5.T1 (with updated need): Need' (0,3,3,0) ≤ Work (2,10,10,12) → **Yes**.

`1.Work ← (2,10,10,12) + Alloc'(T1) (1,4,2,0) = (3,14,12,12)`

`2.Finish = [T,T,T,T,T]`

3. Final seq:< **T0 ,T2 , T3 , T4,T1>**

All processes finish in the tentative state → **the request can be granted immediately** and the system remains safe.

Processes	Allocation A B C	Max A B C	Available A B C
P0	1 1 2	4 3 3	2 1 0
P1	2 1 2	3 2 2	
P2	4 0 1	9 0 2	
P3	0 2 0	7 5 3	
P4	1 1 2	1 1 2	

1. calculate the content of the need matrix?
2. Check if the system is in a safe state?
3. Determine the total sum of each type of resource?
4. Additional Resource Request from process P2(1,0,0), can this be satisfied?